

SAC-TOPSIS: A Decision Model for Task Offloading in Vehicular Networks

Antônio Sérgio de Sousa Vieira^{1,2*}, Alisson Barbosa de Souza^{1,3} and Joaquim Celestino Júnior¹

^{1*}Intelligent Networks and Systems Group (INETSYS), State University of Ceará, Fortaleza, Brazil.

²Federal Institute of Education, Science and Technology, Itapipoca, Brazil.

³Federal University of Ceará, Quixadá, Brazil.

*Corresponding author(s). E-mail(s): sergio.vieira@ifce.edu.br;
Contributing authors: alisson@ufc.br; joaquim.celestino@uece.br;

Abstract

Task offloading in Vehicular Edge Computing (VEC) networks is a critical mechanism to satisfy the stringent latency and energy constraints of advanced autonomous applications. High vehicle mobility creates an environment where the number of available edge servers and neighboring vehicles changes continuously, making the observation space dimensionally variable. This variability prevents standard Deep Reinforcement Learning (DRL) agents from generalizing across different vehicular densities without costly retraining. To address this challenge, this article proposes **SAC-TOPSIS**, a hybrid decision-making architecture that integrates the Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS) into the Soft Actor-Critic (SAC) algorithm. The TOPSIS module evaluates each offloading candidate node, characterized by Job Completion Time (JCT), energy consumption, and physical distance, using L_1 -Manhattan distances with weights **[0.4; 0.3; 0.3]**, and compresses the variable-sized observation into a fixed 6-dimensional, scale-invariant state vector. A SAC agent receives this compact representation and learns a binary policy: execute locally or offload to the best-ranked candidate. Entropy regularization prevents the policy from collapsing into purely local or purely offloading modes. Extensive simulations in 601 heterogeneous Vehicle-to-Everything (V2X) and 5G scenarios demonstrate that TOPSIS integration consistently improves all tested DRL families: SAC improves from **4.47%** to **44.36%** (+**39.9** pp), Twin Delayed DDPG (TD3) from **41.62%** to **43.01%** (+**1.4** pp), and Proximal Policy Optimization (PPO) from **41.61%** to **41.95%** (+**0.3** pp) in the deadline compliance rate. Among all evaluated policies, SAC-TOPSIS achieves the highest compliance rate (**44.36%**) and the highest Success-Weighted Quality (SWQ) index (**0.1450**). Fundamentally, the model is trained on a single density regime and generalizes zero-shot to unseen vehicular densities from 10 to 30 tasks/s.

Keywords: Task offloading, Vehicular Edge Computing, Soft Actor-Critic, TOPSIS, Multi-criteria state compression, Zero-shot generalization

1 Introduction

Vehicular communication networks have undergone some important technological transitions. Initially, the focus was on Vehicular Ad hoc Networks (VANETs), which prioritized direct connectivity between vehicles but faced challenges related to latency and connectivity due to the high mobility of nodes [1]. With the maturation of the Internet of Things (IoT) and the expansion of embedded sensors in urban infrastructure, a large-scale network paradigm emerged that integrates vehicles, infrastructure, pedestrians, and the cloud, known as the Internet of Vehicles (IoV) ecosystem [1]. This ecosystem was designed to enable truly autonomous and connected Intelligent Transportation Systems (ITS), allowing vehicular applications to simultaneously meet efficiency, reliability, and safety requirements [2].

However, IoV depends on the ability to meet the low-latency and high-throughput requirements of radio technologies. Thus, vehicular communication has undergone several evolutions, starting initially with the allocation of 75 MHz in the 5.9 GHz band for the Dedicated Short-Range Communications (DSRC) standard, which defined the parameters for Vehicle-to-Vehicle (V2V) and Vehicle-to-Infrastructure (V2I) communication in the early VANETs [3]. From this legacy, two technological communication axes were developed. The first, IEEE 802.11bd (Next-Generation V2X), extended IEEE 802.11p by incorporating robustness against fast fading and support for speeds up to 250 km h^{-1} [4]. The second axis focuses on overcoming DSRC's limitations in scalability and deployment cost, which drove the development of Cellular V2X (C-V2X) by the 3GPP [5]. From a regulatory perspective, the dispute for the adoption of these technologies was resolved by the Federal Communications Commission (FCC), which established C-V2X as the mandatory standard for ITS communication, with the official discontinuation of DSRC set for December 2026 [6].

The evolution of communication technologies has succeeded in meeting the network latency and throughput requirements of ITS. However, limitations regarding the computational capacity of embedded devices still needed to be overcome. In this context, Multi-Access Edge Computing (MEC) and its vehicular specialization, Vehicular Edge Computing (VEC), enable the operation of

latency-sensitive applications by positioning processing and storage resources close to the radio access network [7, 8]. Thus, end users can transfer computationally intensive tasks to edge servers using 5G C-V2X infrastructure. This delegation drastically reduces response time and on-board energy consumption [9, 10, 11], thereby ensuring the Quality of Service (QoS) of latency-sensitive applications.

In vehicular environments, the computational capacities of Interconnected On-Board Units (OBUs) are inherently heterogeneous [12]. This hardware diversity, if well-utilized, enables the formation of cooperative execution environments, in which vehicles equipped with high-performance processing units make their temporary idle capacity available to assist neighboring nodes with computational and energy constraints. The direct sharing of these resources considerably improves the global QoS of ITS [13]. Thus, integrating the high capacity of the edge server with the availability of idle processing from OBUs closes the circle of improvements necessary to enable the execution of computationally intensive, latency-sensitive, and energy-constrained vehicular applications.

The operationalization of this collaborative potential has task offloading as its central mechanism. This strategy consists of transferring the responsibility for computing complex tasks from resource-limited devices to more powerful remote processing nodes, such as edge servers or neighboring vehicles with greater processing capacity [14]. The goal is to bypass local battery, storage, and processing constraints to reduce energy consumption, minimize latency, and improve QoS. The great challenge, however, lies in dynamically deciding when and where to offload these tasks. Fully exploring this potential requires the development of allocation policies capable of adapting to the highly dynamic network topology and evaluating multiple decision criteria simultaneously and in real-time.

Task offloading in vehicular networks is fundamentally a multi-objective optimization problem [15, 16]. In this context, various policies have been proposed to solve this problem. Heuristics based on logical rules and Multi-Criteria Decision Making (MCDM) methods, such as greedy

strategies, are widely used due to their computational simplicity and ease of implementation [17]. However, these approaches only evaluate the instantaneous network situation, proving incapable of anticipating traffic dynamics and future states of the environment. On the other hand, meta-heuristics, such as genetic algorithms, particle swarm optimization, and ant colony optimization, offer more solid global search guarantees and have been successfully applied to offloading decision-making in static scenarios [18, 19, 20]. However, these methods may prove inadequate for vehicular environments, as population-based search requires tens to hundreds of iterative fitness evaluations at each decision point, which can cause unacceptable delays [21, 22]. This relatively slow convergence characteristic can be incompatible with the rigorous execution deadlines on the millisecond scale characteristic of vehicular applications [8, 2]. Furthermore, these algorithms need to be restarted with each topological change in the environment, and their initialization and convergence complexity grows significantly with the increase in network size, potentially degrading the quality of the solution in large-scale scenarios [15, 11].

Simplified evaluation scenarios, with a fixed number of vehicles, uniform task distributions, and idealized channel models, do not capture the complexity and high dynamism of real vehicular networks [8]. Offloading policies trained and evaluated in these environments can suffer performance degradation when vehicular density or workload distribution changes suddenly [8], revealing a critical dichotomy between the success obtained in simulations and real applicability in operational scenarios. In this context, it is fundamental that the development of robust offloading policies actively seeks to mitigate this disparity. This requires the design of adaptive state representations that ensure the model’s generalization capability in the face of constant changes in the number of candidate nodes for offloading, eliminating the dependence on stationary traffic environments [23, 2].

DRL presents itself as a natural solution for task offloading because it learns a control policy that maps observations to actions considering long-term future consequences, rather than just the instantaneous state of the network [9, 24]. Unlike heuristics and meta-heuristics, a DRL agent has the ability to explore temporal correlations

in traffic patterns, queue dynamics, vehicular mobility, and communication channel quality from continuous interaction with the environment [25, 2].

However, standard DRL architectures share a critical structural weakness related to their practical applicability, as they frequently require retraining whenever the operational scenario undergoes changes in the number of candidate computational nodes [8]. Since the input layer of deep neural networks depends on a fixed-size vector, constant changes in the number of candidate nodes for offloading, such as the edge server and neighboring vehicles, end up invalidating the previously trained model [2].

In this context, this work proposes a hybrid task offloading policy called SAC-TOPSIS capable of handling the operational constraints of DRL in a dynamic vehicular environment. In summary, the main contributions are:

1. Development of an innovative solution for task offloading that combines the TOPSIS technique with the SAC algorithm. TOPSIS provides the best alternative based on instantaneous network conditions, while SAC learns the long-term optimal policy through accumulated experience.
2. Representation of a variable-sized set of candidate nodes for offloading in a compact six-dimensional state vector, allowing the control policy trained in a representative scenario to be tested in unseen environments without the need for retraining.
3. Reduction of the action space to a binary decision (local processing vs. best offloading candidate), enabling inference in $O(1)$ direct passes through the neural network per task, making the model suitable for real-time embedded deployment.
4. Development of a deadline-sensitive reward function that provides a continuous gradient signal in both success and failure regimes, avoiding reward sparsity and teaching the agent to actively minimize delay rather than merely avoid it.

The remainder of this article is organized as follows. Section 2 reviews related work in the literature. Section 3 formalizes the system model and defines the optimization objective. Section 4 details the proposed SAC-TOPSIS hybrid policy

architecture. Section 5 presents the experimental evaluation and discusses the results obtained. Section 6 concludes the article, presenting final considerations and directions for future work. Furthermore, the definitions of all acronyms used throughout the text are summarized in Table 1.

2 Related Work

The optimization of task offloading in vehicular networks represents a critical challenge for enabling intelligent transportation systems, requiring fast decisions that balance latency and energy consumption. Advances in Vehicular Edge Computing (VEC) architectures have moved processing capacity closer to users, but high mobility and varying network loads make static heuristics inefficient. Consequently, recent literature has transitioned from traditional mathematical methods to Deep Reinforcement Learning (DRL) approaches, seeking allocation policies that adapt dynamically to the environment.

The work presented in [26] developed the FOFF framework based on Multi-Criteria Decision Making (MCDM). The solution applied the Fuzzy-TOPSIS technique to rank edge servers (VEC) and neighboring vehicles based on response time and energy consumption. Simulations demonstrated that the approach achieved significant energy savings without the need for a prior training phase. However, models based exclusively on logical rules or heuristic evaluations fail in non-stationary vehicular scenarios, as they cannot anticipate fluctuations in queue dynamics or learn from performance history.

The study conducted by [27] explored machine learning to handle user mobility in vehicular networks. The research proposed a framework based on Deep Q-Networks (DQN) to decide allocation at roadside units and manage task offloading. The results indicated an effective reduction in communication and computation costs. The main limitation of this approach lies in its centralized nature, requiring simultaneous control of all vehicles in the network and causing an exponential increase in computational complexity as vehicular density grows.

The work by [28] developed a multi-task offloading algorithm using the Dueling Double Deep Q-Network (D3QN) architecture. The method focused on minimizing response time and energy

consumption in highly requested edge environments. Evaluations showed that the model reduced system latency more effectively than traditional random approaches. However, being based on strictly discrete action spaces, the algorithm faces scalability difficulties when applied to continuous resource allocation problems, limiting the precision of computational power distribution.

Research presented by [29] proposed an offloading solution based on a four-stage Stackelberg game integrated with a reinforcement learning model. The strategy evaluates vehicle state in real-time to optimize offloading time and expense in a coordinated manner. Experiments confirmed that the solution achieved balance in system utility and improved task success probability. Despite the gains, reliance on complex negotiation processes between multiple agents introduces signaling overhead that degrades performance in dense traffic and intermittent communication scenarios.

The study by [24] sought to solve the problem of continuous resource allocation by applying the Deep Deterministic Policy Gradient (DDPG) algorithm. The approach used an actor-critic structure to reduce overall system cost in a collaborative cloud and vehicular edge environment. The algorithm converged quickly and reduced operational costs compared to value-only methods. The failure of this modeling is the lack of consideration for structural dependencies between tasks and the use of zero-padding in state observations to handle a dynamic number of vehicles, forcing the neural network to process spurious artifacts and noise.

The work developed by [30] implemented an adaptive scheme using multi-agent reinforcement learning in vehicular fog computing. The solution distributed decision-making directly among vehicles to reduce latency and energy consumption in a decentralized manner. Simulations demonstrated superiority in local adaptation capacity compared to single-agent guided architectures. The critical limitation of this architecture is the synchronization challenge and the difficulty of ensuring stable convergence when multiple vehicles change the environment simultaneously, generating instability in the global routing policy.

The study conducted by [31] integrated digital twin technology with vehicle networks to enable proactive artificial intelligence-assisted offloading. The work employed the Asynchronous Advantage

Table 1 List of Acronyms – Summary of the main acronyms used throughout this article.

Acronym	Definition	Acronym	Definition
3GPP	3rd Generation Partnership Project	NIS	Negative Ideal Solution
5G	5th Generation	NR-V2X	New Radio Vehicle-to-Everything
C-V2X	Cellular Vehicle-to-Everything	OBUI	Interconnected On-Board Unit
D3QN	Dueling Double Deep Q-Network	PIS	Positive Ideal Solution
DRL	Deep Reinforcement Learning	QoS	Quality of Service
DSRC	Dedicated Short-Range Comm.	RSU	Roadside Unit
IEEE	Inst. of Elect. and Electr. Eng.	SAC	Soft Actor-Critic
IoT	Internet of Things	TOPSIS	Tech. Order Pref. Sim. Ideal Sol.
IoV	Internet of Vehicles	V2I	Vehicle-to-Infrastructure
ITS	Intelligent Transportation Systems	V2N	Vehicle-to-Network
JCT	Job Completion Time	V2P	Vehicle-to-Pedestrian
MCDM	Multi-Criteria Decision Making	V2V	Vehicle-to-Vehicle
MEC	Multi-Access Edge Computing	VEC	Vehicular Edge Computing

Actor-Critic (A3C) algorithm to optimize decisions based on a continuous virtual replica of the physical vehicular environment. The research reported rapid network convergence and considerable reduction in system cost. The fundamental restriction of this model is the extremely high computational overhead required to keep the digital twin synchronized in real-time, a factor that frequently makes its smooth execution on mobile devices unfeasible.

Research by [32] directed reinforcement learning toward computer vision tasks and proposed an algorithm focused on object detection in autonomous vehicles. The model combined DRL with piecewise linearization to specifically maximize a utility function based on execution time. The results pointed to significant improvements in response time for critical image processing. However, the modeling was built monolithically for a specific type of application, failing to consider the priorities of different competing tasks and neglecting congestion issues caused by vehicular density.

The study presented by [33] focused on offloading complex tasks modeled as directed acyclic graphs using the Soft Actor-Critic (SAC) algorithm. The framework sought to actively balance delay and energy expenditure for applications with high structural dependency between their subcomponents. Evaluation attested that stochastic entropy maximization avoided convergence to local optima and improved execution utility. The main restriction of this model for the automotive context is the premise of a perfectly static environment, completely ignoring disconnection rates and communication channel variability.

A hybrid approach was developed by [21], which proposed the TOLB scheme for offloading and load balancing by integrating Twin Delayed Deep Deterministic Policy Gradient (TD3) with the Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS). The methodology used the learning algorithm for partitioning proportion decisions and multi-criteria calculation to select the target server. Experiments proved the reduction of processing delay and decrease in load imbalance in the edge infrastructure. The limitation of this approach lies in using multi-criteria filtering only as a tactical post-processing element, keeping the observation space raw and rigid at the agent’s input and hindering spatial generalization in dense networks.

The proposed SAC-TOPSIS architecture in this intermediate stage differs substantially from these approaches, as it consolidates the TOPSIS method internally within the observation edge, acting strictly as a state engineering module. This module evaluates the edge server (VEC) and neighboring vehicles based on Job Completion Time (JCT), energy consumption, and distance to the computing node to identify the best offloading candidate. The computed output condenses these characteristics into a strictly compact continuous state encoding as a 6D vector that is perfectly scale-invariant, eliminating the dependence on zero-padding. The experimental evaluation compared five DRL families (SAC, Proximal Policy Optimization (PPO), TD3, DQN, and Advantage Actor-Critic (A2C)) and seven baseline heuristics (VEC, Random, Round-Robin, Gini, Jain, Greedy-Fair, and Greedy-Unfair), confirming that pairing this structured space yields

consistent gains. This validates the central thesis that compositional state engineering developed via TOPSIS acts as the primary driver for the generalization capability exhibited by the intelligent agent in variable traffic environments. In summary, the main aspects of related works are presented in Table 2.

3 System Model and Problem Formulation

This section presents the main mathematical models that support the proposed solution, encompassing task offloading and latency dynamics, thermal dynamics (thermal throttling and cooling), energy consumption, cellular and vehicular communication, and the definition of the objective function. To facilitate reading and formal understanding, Table 3 summarizes the main notations used.

3.1 Task Model, Latency, and Thermal Dynamics

Each task is characterized by the pair $\mathcal{T}_i = (c_i, d_i)$, where c_i represents the task complexity (in CPU cycles) and d_i denotes the acceptable deadline for its completion [9, 2, 34, 22]. The task offloading decision is binary (local or remote processing) with the goal of reducing the computational complexity of the control policy, which favors its applicability in real-time systems [9, 29].

The execution latency is modeled by the ratio between the total workload (c_i) and the processing capacity of the device (f^m). Consequently, the response time for processing task i in the selected mode $m \in \{local, remote\}$ is expressed by Equation 1, and the total job completion time (JCT) is given by Equation 2:

$$L_i^m = \frac{c_i}{f^m} \quad (1)$$

$$JCT_i = t_i^{\text{upload}} + t_i^{\text{queue}} + L_i^m + t_i^{\text{download}}, \quad (2)$$

where t_i^{upload} is the transmission time of task i , t_i^{queue} is the residual waiting time in the processing queue (estimated as the remaining execution time of already scheduled tasks), L_i^m is the execution time of the task in the selected mode m , and

t_i^{download} is the time required to download the results. For the local processing mode, the upload and download transmission times are zero.

To make the JCT calculation more realistic, it is necessary to consider the impact of the device's thermal security mechanism. The continuous execution of tasks raises the processor temperature, which may force the system to reduce the operating frequency f^m to avoid physical overheating of the device.

In this context, the electrical power dissipated by the CPU ($P_{\text{cpu}}(f^m)$) is calculated using the frequency polynomial model proposed by Silva et al. [35], given by Equation 3:

$$P_{\text{cpu}}(f^m) = k_1(f^m)^3 + k_2f^m + k_3 \quad (3)$$

where k_1 and k_2 represent the dynamic and leakage power constants, respectively. The constant k_3 indicates the minimum static consumption of the architecture.

The instantaneous processor temperature $T_p(t)$, after a time interval under a constant power dissipation $P_{\text{cpu}}(f^m)$, follows the RC thermal circuit model by Rao and Vrudhula [36], expressed by Equation 4:

$$T_p(t) = P_{\text{cpu}} \cdot R'_p + T_{\text{amb}} + (T_{p0} - (P_{\text{cpu}} \cdot R'_p + T_{\text{amb}}))e^{-\frac{t}{R'_p C_p}} \quad (4)$$

where T_{p0} is the initial temperature, T_{amb} is the ambient temperature, and R'_p and C_p are parameters representing the thermal resistance and capacitance of the CPU, respectively.

This modeling allows for a more realistic representation of the processor's heating and cooling cycles. In the heating state, the required power ($P_{\text{cpu}} = P'_{\text{max}}$) is high, generating an exponential increase in temperature. To protect the processor, the model mathematically projects the exact instant t_m when the temperature will reach the critical limit ($T_{p,\text{max}}$). Deriving Equation 4, the safety constraint is obtained (Equation 5):

$$t_m = R'_p C_p \ln \left(\frac{P'_{\text{max}} R'_p + T_{\text{amb}} - T_{p0}}{P'_{\text{max}} R'_p + T_{\text{amb}} - T_{p,\text{max}}} \right) \quad (5)$$

Table 2 Comparison of Offloading Solutions – A comparative evaluation of offloading strategies proposed in previous studies.

Ref.	Strategy	Communication Technology	Achievement	Limitation
[26]	Fuzzy-TOPSIS	VEC	Significant energy savings without prior training phase	Fails to adapt to non-stationary vehicular scenarios
[27]	DQN	Vehicular Networks	Effective reduction in communication and computation costs	Exponential increase in complexity due to centralized nature
[28]	D3QN	IoV and MEC	More effective system latency reduction	Scalability difficulties in continuous allocation problems
[29]	DRL + Stackelberg Game	Vehicular Networks	System utility and task success probability balance	High signaling overhead in dense traffic scenarios
[24]	DDPG	Vehicular Cloud and Edge	Fast convergence and operational cost reduction	Use of zero-padding and noise processing
[30]	MARL	Vehicular Fog	Superiority in decentralized local adaptation capacity	Synchronization challenge and unstable global network convergence
[31]	A3C + Digital Twins	IoV	Fast network convergence and system cost decrease	Extremely high computational overhead for synchronization
[32]	DRL + Piecewise linearization	Autonomous Vehicles	Significant response time improvements for image processing	Neglects congestion issues and task priorities
[33]	SAC	Vehicular Cloud	Avoids local optima convergence and improves utility	Static environment premise and channel variability ignorance
[21]	TD3 + TOPSIS	VEC	Processing delay and load imbalance reduction	Multi-criteria filtering used only as tactical post-processing

If the processing time exceeds t_m , the temperature will inevitably reach the maximum boundary $T_{p,\max}$. At this threshold, the thermal containment mechanism (thermal throttling) is triggered, forcing the reduction of the peak frequency f^m to a safe thermal stabilization frequency, slowing down the heat dissipation curve at the cost of significantly extending the latency L_i^m .

When the processing queue is empty, the CPU ceases computational activity and f^m becomes

virtually zero. In this state, the dissipated power drops from $P'_{\max}$ to $P_{\text{cpu}} = k_3$, which represents only the static consumption of the architecture. Inserting this minimum subtractive power into Equation 4, the asymptotic temperature drops drastically, making the cooling differential $(T_{p0} - (k_3 \cdot R'_p + T_{\text{amb}}))$ strongly positive. Therefore, the exponential decay acts as a buffer, continuously reducing the temperature $T_p(t)$ toward the new safe

Table 3 Summary of Used Notations

Symbol	Description
<i>Task Model</i>	
T_i	Task i defined by (c_i, d_i)
c_i	Computational complexity of task i (in cycles)
d_i	Maximum deadline of task i (in s)
m	Execution mode (l : local, r : remote)
f^m	CPU operating frequency in mode m (in Hz)
<i>Thermal Dynamics and Energy</i>	
P_{cpu}^m	Electrical power consumed by the CPU (in W)
k_1, k_2, k_3	Power coefficients (dynamic, leakage, and static)
$T_p(t)$	Chip temperature at time t (in $^{\circ}\text{C}$)
T_{amb}	External ambient temperature (in $^{\circ}\text{C}$)
$T_{p,\text{max}}$	Critical temperature limit (throttling) (in $^{\circ}\text{C}$)
R'_p, C_p	Thermal resistance ($^{\circ}\text{C W}^{-1}$) and capacitance ($\text{J }^{\circ}\text{C}^{-1}$)
<i>Metrics and Optimization</i>	
L_i^m	Task processing time in mode m (in s)
JCT_i	Total job completion time (in s)
E_i^m	Total energy consumed to process task i (in J)
Φ_i	QoS contribution of task i
α, β, γ	Weights for latency, energy, and deadline penalty

basal rest threshold throughout the idle interval time.

3.2 Energy Model

Thus, the total energy consumed E_i^m during the entire life cycle of task i in mode $m \in \{l, r\}$ coherently encompasses the instantaneous processing power and the cost of network interfaces, according to Equation 6:

$$E_i^m = P_{\text{cpu}}^m(f^m) \cdot L_i^m + P_{\text{tx}}^m \cdot t_i^{\text{trans},m} \quad (6)$$

where P_{tx}^m denotes the constant transmission power and $t_i^{\text{trans},m}$ represents the total transmission time of the task (encompassing upload and download). For the local processing mode ($m = l$), the network interface is not activated, making the transmission term strictly zero ($P_{\text{tx}}^l = 0$).

3.3 Communication Model (V2I and V2V)

The modeling of V2I and V2X communication follows the definitions from [37, 38]. The V2I link operates at a frequency of 3.5 GHz with a bandwidth of 50 MHz, a transmission power of 23 dBm, and asymmetric antennas (1.5 m on vehicles and 25 m at the base station). The fading value adopted is 4 dBm with a processing delay (jitter) of 0.1 ms. For the V2X interface (Sidelink), the center frequency is 5.9 GHz, with a bandwidth of 10 MHz, fading of 3 dBm, and a processing delay of 0.25 ms. Both interfaces adopt the 3GPP Urban Non-Line-of-Sight (Urban NLOS) model [37], where the path loss (**PL**) expressed in dB is determined by Equation 7:

$$PL(d, f) = 35.3 \log_{10}(d) + 21.3 \log_{10}(f) + 22.4 \quad (7)$$

where d represents the three-dimensional distance between the transmitter and receiver in meters, and f is the operating frequency in GHz. These parameters ensure a realistic representation of signal attenuation during data exchange between candidate offloading nodes, such as the edge server (VEC) and neighboring vehicles.

3.4 Optimization Objective

The formulation of the QoS contribution for task i is expressed by Equation 8:

$$\Phi_i = \alpha \cdot \text{JCT}_i + \beta \cdot E_i + \gamma \cdot \mathbf{1}[\text{JCT}_i > d_i], \quad (8)$$

which incorporates configurable weight parameters (by default, $\alpha = \beta = 0.35$) and a strict penalty constant for deadline violation ($\gamma = 0.015$). The global goal of the system is to minimize the aggregate cost $\sum_i \Phi_i$ while actively maximizing the deadline fulfillment rate, defined as $\hat{\tau} = |\{i : \text{JCT}_i \leq d_i\}|/N$.

4 The SAC-TOPSIS Architecture

Figure 1 illustrates the complete architecture. At each decision point, the TOPSIS module processes

SAC-TOPSIS Architecture: TOPSIS State Compression + Maximum-Entropy Policy

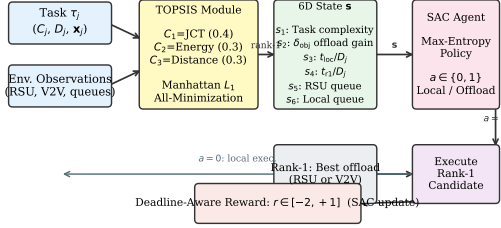


Figure 1 SAC-TOPSIS Architecture. TOPSIS compresses the variable-sized set of candidates into a fixed 6D scale-invariant state vector. SAC learns a binary policy (local vs. best offloading) over this representation using maximum entropy regularization.

multi-criteria information about all viable candidates and produces the 6D state vector passed to the SAC agent.

4.1 TOPSIS-Based State Compression

Decision matrix.

For task τ_j , we construct a decision matrix $\mathbf{X} \in \mathbb{R}_{\geq 0}^{3 \times 3}$ whose rows correspond to the three candidate modes (local, RSU, V2V) and whose columns are the three cost criteria:

$$\mathbf{X} = \begin{bmatrix} \text{JCT}_j^{(0)} & E_j^{(0)} & 0 \\ \text{JCT}_j^{(1)} & E_j^{(1)} & d_{\text{RSU}} \\ \text{JCT}_j^{(2)} & E_j^{(2)} & d_{\text{V2V}} \end{bmatrix}, \quad (9)$$

where d_{RSU} and d_{V2V} are the Euclidean distances to the RSU and the selected V2V pair, respectively. If a mode is unfeasible, its row is set to 10^6 to exclude it from the ideal solutions.

Weighted normalization.

Each column j is normalized by its Euclidean norm $\|\mathbf{X}_{:,j}\|_2$ and then multiplied by the weight $w_j \in \{0.4; 0.3; 0.3\}$ (JCT, energy, distance), resulting in the weighted normalized matrix $\tilde{\mathbf{X}}$.

TOPSIS L_1 -Manhattan.

We differ from classic TOPSIS by measuring distances to the positive ideal solution (PIS) A^+ and the negative ideal solution (NIS) A^- using the

Manhattan norm (L_1):

$$D_i^+ = \sum_k |\tilde{x}_{ik} - A_k^+|, \quad D_i^- = \sum_k |\tilde{x}_{ik} - A_k^-|. \quad (10)$$

The closeness score is then $\mathcal{C}_i = D_i^- / (D_i^+ + D_i^-)$. Since all criteria are cost minimization objectives, A^+ is the column-wise minimum and A^- is the column-wise maximum of $\tilde{\mathbf{X}}$. The L_1 metric is less sensitive to extreme values, a critical property when queue estimates are noisy, and is computationally lighter than L_2 .

Rank-1 selection.

The rank-1 offloading candidate is the feasible non-local mode with the highest $\mathcal{C}_i$:

$$m^* = \underset{i \in \{1,2\}}{\operatorname{argmax}} \mathcal{C}_i \text{ s.t. mode } i \text{ is feasible.} \quad (11)$$

Ties are resolved in favor of V2V to encourage load balancing. If no offloading mode is feasible, m^* reverts to local execution regardless of the agent's action.

6D state vector.

The state $\mathbf{s} \in \mathbb{R}^6$ is assembled as:

$$\begin{aligned} s_1 &= C_j / 10^{10} \\ s_2 &= \operatorname{clip}\left(1 - \frac{\operatorname{cost}_{m^*}}{\operatorname{cost}_{\text{loc}}}, -1, 1\right) \\ s_3 &= \min\left(1, \text{JCT}^{(0)} / D_j\right) \\ s_4 &= \min\left(1, \text{JCT}^{(m^*)} / D_j\right) \\ s_5 &= \min(1, |Q_{\text{RSU}}| / 25) \\ s_6 &= \min(1, |Q_{v_0}| / 25) \end{aligned} \quad (12)$$

where the components represent: task complexity; offloading advantage; local urgency; rank-1 urgency; RSU queue occupancy; and local queue occupancy. where $\operatorname{cost}_m = \alpha \text{JCT}^{(m)} + \beta E^{(m)}$. All components belong to $[0, 1]$ or $[-1, 1]$, making $\mathbf{s}$ scale-invariant, as it does not depend on the number of available candidates, only on the relative quality of the best one.

4.2 Maximum Entropy Soft Actor-Critic

The SAC agent optimizes the maximum entropy objective [39]:

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \mathbb{E}_{\tau \sim \pi} \left[\sum_t \gamma^t (r_t + \alpha_T \mathcal{H}[\pi(\cdot | s_t)]) \right], \quad (13)$$

where α_T is the automatically adjusted temperature parameter. The actor and critic networks are MLPs with two hidden layers of 256 units and ReLU activations. The actor generates logits for the Bernoulli action distribution; the temperature α_T is adapted to a target entropy of $\log 2/2$ nats. The replay buffer stores 10^5 transitions; the mini-batch size is 256; the learning rate is 3×10^{-4} for all networks.

Why binary action?

Delegating the selection of the offloading destination to TOPSIS reduces the action space from $|\mathcal{V}(t)| + 2$ dimensions to a single bit. This simplification is lossless from the agent’s perspective, as the relative ordering of candidates is already encoded in s_2 – s_4 .

4.3 Deadline-Aware Reward Function

The reward r_j for task τ_j is designed to guide the agent toward deadline fulfillment while providing a continuous gradient signal:

$$r_j = \begin{cases} 1.0 - 0.5 \cdot \rho_j & \text{if } \text{JCT}_j \leq D_j, \\ -1.0 - \min(1, \delta_j) & \text{otherwise,} \end{cases} \quad (14)$$

where

$$\rho_j = \frac{1}{2} \left(\frac{\text{JCT}_j}{D_j} + \frac{E_j}{E_{\max,j}} \right) \in [0, 1] \quad (15)$$

is a normalized cost ratio, and $\delta_j = (\text{JCT}_j - D_j)/D_j$ is the relative deadline violation. Successful executions yield $r_j \in [0.5, 1.0]$: faster and more efficient tasks receive higher rewards. Failed executions yield $r_j \in [-2, -1]$: the penalty grows with the degree of delay, providing a gradient that teaches the agent to fail for the shortest possible time.

5 Performance Evaluation

5.1 Simulation Setup

All algorithms run on a custom discrete-event VEC simulator written in C++20 using `simcpp20` and the Armadillo linear algebra library. Neural networks are trained and exported using LibTorch (PyTorch’s C++ API). Mobility scenarios are generated with realistic V2X parameters, with vehicular speeds from 30 km h^{-1} to 120 km h^{-1} , 5G uplink bandwidth of 20 MHz, V2V sidelink of 10 MHz, RSU frequency $f_{\text{RSU}} = 2 \text{ GHz}$, and on-board frequency $f_{\text{loc}} \in [0.5; 1.0] \text{ GHz}$. Task deadlines follow a truncated Gaussian in $[0.05; 2.0] \text{ s}$; CPU complexity $C_j \sim \text{Uniform}[10^8, 10^{10}]$ cycles.

Training.

All DRL agents are trained for 400 episodes in scenarios with a fixed vehicular density of 10 tasks/s. Each training episode simulates 100 task arrivals with a new mobility realization drawn from the same distribution.

Testing.

After training, agents are evaluated zero-shot across 601 test scenarios covering five density regimes: $\{10, 15, 20, 25, 30\}$ tasks/s. No additional learning or adaptation occurs during testing.

Baselines.

We compare against: Always-Local (never offloads), Greedy (always offloads to the V2V pair with the lowest estimated JCT, an optimistic oracle baseline), Greedy-Fair (greedy with Jain’s fairness constraint), Random (offloads to a random pair), SAC-Base (SAC with an 11-dimensional raw state vector, without TOPSIS), PPO-TOPSIS (on-policy PPO with the same 6D TOPSIS state), and TD3-TOPSIS (off-policy TD3 with the same 6D TOPSIS state).

Metrics.

- **Deadline fulfillment rate:** fraction of tasks with $\text{JCT}_j \leq D_j$.
- **SWQ:** $\text{SWQ} = \hat{\tau} \cdot \overline{[(D_j - \text{JCT}_j)/D_j]_{\text{success}}}$, which rewards meeting the deadline with a large margin.
- **Average JCT and total energy consumed.**

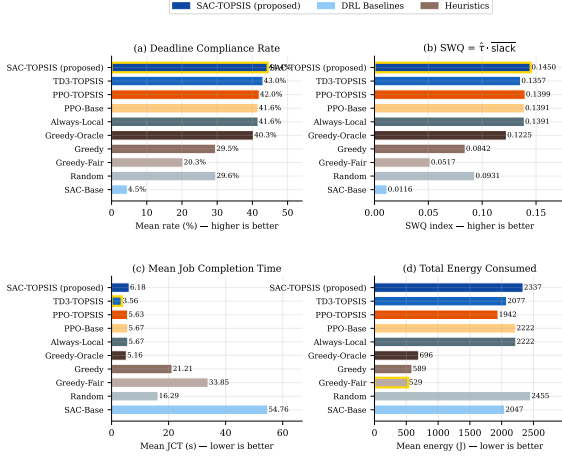


Figure 2 Performance comparison across main metrics. SAC-TOPSIS (dark blue) achieves the highest deadline fulfillment rate and the highest SWQ index. The gold border marks the best value in each chart.

5.2 General Performance Comparison

Figure 2 and Table 4 summarize the aggregated performance across all 601 test scenarios.

TOPSIS consistently improves all DRL families.

Table 4 reveals a central result of this article: pairing TOPSIS with any tested DRL algorithm improves the deadline fulfillment rate relative to its corresponding base version. The magnitude of the gain, however, varies substantially across algorithm families and reflects the interaction between the TOPSIS state and each algorithm’s exploration strategy.

SAC experiences the most dramatic improvement: SAC-Base achieves only 4.47% fulfillment because it converges to an almost-always-offload policy (94.9% V2V), incurring prohibitive queue delays when neighbors are congested. Adding the TOPSIS state rescues the policy to 44.36% (+39.9 pp), the best result among all evaluated policies. The 6D representation provides the semantic signal, with urgency ratios and queue occupancies, necessary for SAC to distinguish beneficial from harmful offloading decisions.

TD3 already collapses to always-local in its base form (41.62%, identical to Always-Local), as its deterministic policy avoids the risky V2V path without exploration pressure. With the TOPSIS

Table 4 Aggregated performance across 601 test scenarios (average per scenario, densities {10, 15, 20, 25, 30} tasks/s). Arrows ($\uparrow$) indicate the gain from adding TOPSIS to each base algorithm. Bold indicates the best DRL value; $\dagger$ indicates the oracle baseline.

Policy	Deadline (%)	SWQ	JCT (s)	Energy (J)
SAC-Base	4.47	0.0116	54.76	2047
SAC-TOPSIS	44.36	0.1450	6.18	2337
$\uparrow$ +39.9				
TD3-Base	41.62	0.1391	3.53	2222
TD3-TOPSIS	43.01	0.1357	3.56	2077
$\uparrow$ +1.4				
PPO-Base	41.61	0.1391	5.67	2222
PPO-TOPSIS	41.95	0.1399	5.63	1942
$\uparrow$ +0.3				
Always-Local	41.61	0.1391	5.67	2222
Greedy-Oracle $\dagger$	40.31	0.1225	5.16	696
Greedy	29.54	0.0842	21.21	589
Greedy-Fair	20.26	0.0517	33.85	529
Random	29.61	0.0931	16.29	2455

state, TD3 learns to exploit V2V opportunistically (23.4%), raising fulfillment to 43.01% (+1.4 pp) and achieving the lowest average JCT (3.56 s) of all policies.

PPO also collapses to always-local in its base form (41.61%) due to the high variance of the on-policy gradient. The TOPSIS state provides a marginal improvement to 41.95% (+0.3 pp). The gain is limited because PPO’s on-policy nature prevents it from fully exploiting the richer state, as variance in gradient estimates still pushes the policy toward the safe local action despite the informative s_2 signal (offloading advantage).

Overall, SAC-TOPSIS achieves the highest deadline fulfillment rate (44.36%) and SWQ (0.1450) among all policies, including the Greedy oracle baseline (40.31%). The results confirm that the TOPSIS state is the primary performance

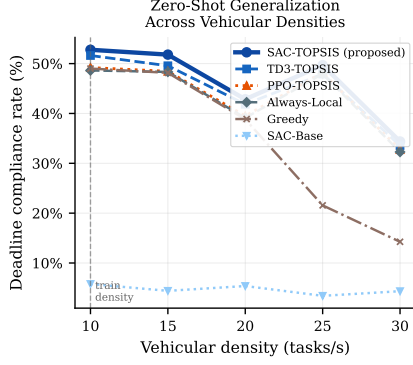


Figure 3 Deadline fulfillment rate as a function of vehicular density. All agents were trained only at 10 tasks/s (dashed vertical line). SAC-TOPSIS (solid circles) consistently outperforms all baselines without density-specific retraining.

driver, rather than the choice of DRL algorithm, as all three families benefit, and the residual differences between them are side effects of their respective exploration mechanisms.

5.3 Zero-Shot Generalization

Figure 3 shows the deadline fulfillment rate as a function of vehicular density. SAC-TOPSIS consistently leads across all densities, maintaining $> 34\%$ fulfillment even in the most severe unseen congestion regime (30 tasks/s), where heuristic Greedy drops to only 14%. The difference over Always-Local ranges from +1.73 pp at 25 tasks/s to +4.15 pp at 10 tasks/s, confirming that the benefit of learning an adaptive policy is more pronounced in lower congestion, where opportunistic V2V offloading is more reliable.

The scale invariance of the 6D state vector is the fundamental enabler: as s_3-s_4 are deadline-relative ratios and s_5-s_6 are normalized queue occupancies, the policy learned by the agent transfers directly to environments with more or fewer candidates. SAC-Base, in contrast, encodes absolute queue sizes and raw frequency values that lose their meaning under distribution changes.

5.4 Execution Mode Distribution

Figure 4 shows the average proportion of local, RSU, and V2V executions per policy. SAC-TOPSIS exploits V2V offloading (25.9%), which offers lower latency than RSU when a peer vehicle is close. Crucially, the TOPSIS module ensures

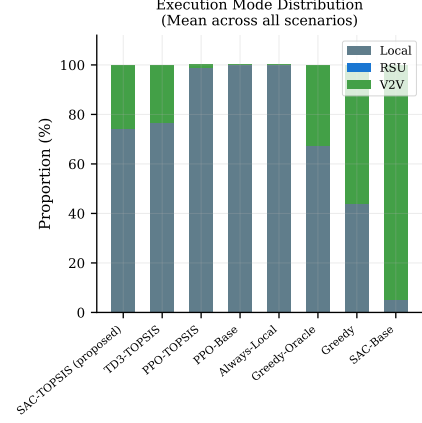


Figure 4 Execution mode distribution. SAC-TOPSIS adaptively balances local and V2V execution, while SAC-Base overloads the RSU with offloads.

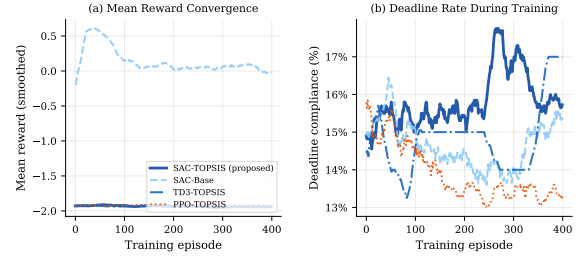


Figure 5 Training convergence over 400 episodes. (a) Average reward (smoothed with a 20-episode window); (b) deadline fulfillment rate during training. SAC-TOPSIS achieves stable convergence in both metrics.

that V2V is chosen only when it offers a better TOPSIS score than RSU, preventing load migration to overloaded peers.

5.5 Training Convergence

Figure 5 compares the training curves of SAC-TOPSIS, SAC-Base, TD3-TOPSIS, and PPO-TOPSIS. All agents are trained for 400 episodes in the 10 tasks/s scenario. SAC-TOPSIS and TD3-TOPSIS exhibit faster reward improvement and lower variance than SAC-Base, confirming that the TOPSIS state provides a more informative learning signal. PPO-TOPSIS converges to a local optimum with lower average reward, consistent with its always-local policy at test time.

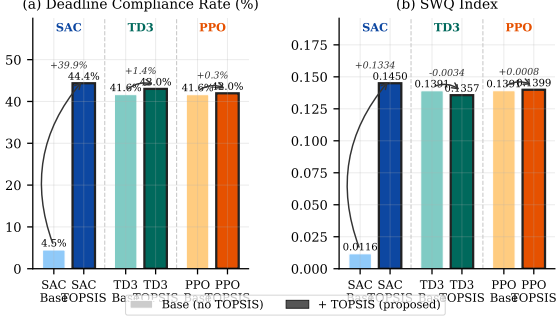


Figure 6 Cross-family ablation: Base (without TOPSIS, lighter bars) vs. +TOPSIS (darker bars with black border) for SAC, TD3, and PPO. Curved arrows show the gain from adding TOPSIS to each family. The gain is consistent across all families, confirming that TOPSIS state compression is a universal performance driver.

5.6 Ablation Study: TOPSIS as a Universal State Engineering Primitive

Figure 6 presents a cross-family ablation in which each DRL algorithm is evaluated with its raw base state and with the 6D TOPSIS state, keeping all other hyperparameters identical. The arrows annotating each pair quantify the gain attributable exclusively to the change in state representation.

Three findings stand out. First, the gain is monotonic, as TOPSIS never harms any algorithm. Second, the gain is larger for the most unstable base policy (SAC-Base collapses, SAC-TOPSIS recovers), confirming that the TOPSIS state solves the representation collapse problem rather than just fine-tuning a reasonable policy. Third, the gain is algorithm-agnostic, as even TD3, which has a well-behaved (though conservative) base policy, improves by +1.4pp after the state change. This last point is fundamental, as it shows that the TOPSIS state carries useful information beyond what any base algorithm can infer from raw observations, and that this information is exploitable by different learning paradigms.

5.7 Internal SAC Ablation: Causal Decomposition

To understand which design choices within SAC-TOPSIS contribute most to its performance, we conducted a refined ablation in which each variant

Table 5 Internal SAC ablation. Each row adds a design choice over SAC-Base. Δ is the gain in deadline rate relative to SAC-Base.

Variant	Change	Deadline	Δ
SAC-Base	— (base)	4.47%	—
SAC-A1	Binary action	38.59%	+34.1
SAC-R1	Reward	37.64%	+33.2
SAC-S1	Compact state	28.32%	+23.8
SAC-T1	TOPSIS 3-ac	32.10%	+27.6
SAC-TOPSIS	Complete	44.36%	+39.9

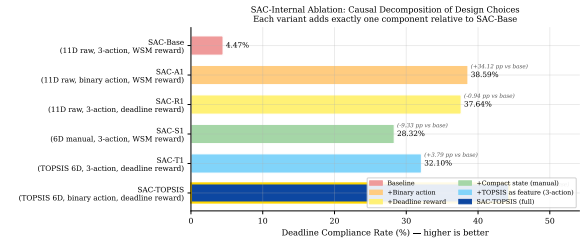


Figure 7 Internal SAC causal decomposition. Each bar corresponds to a variant that isolates a design choice. All gains are reported relative to SAC-Base (4.47%). Complete SAC-TOPSIS combines all three components and achieves the highest fulfillment rate.

adds exactly one component relative to SAC-Base. Table 5 and Figure 7 summarize the results.

Three complementary perspectives emerge from this decomposition.

- (1) Binary action is the largest individual lever (+34.1 pp, SAC-A1). Reducing the action space from three classes (LOCAL/RSU/V2V) to a binary decision (LOCAL vs. rank-1 best offloading) largely solves SAC-Base’s collapse. Without TOPSIS, the agent still must evaluate candidates based on raw features, but simplifying the decision boundary already prevents the degenerate always-V2V policy.
- (2) Deadline-aware reward modeling is equally critical (+33.2 pp, SAC-R1). Replacing Weighted Sum Method (WSM) reward with continuous deadline-proportional reward from Equation (14) provides a gradient signal even in the failure regime, allowing the agent to learn to minimize delay rather than just avoid it. This gain is independent of the state

representation, demonstrating that reward engineering is an autonomous contribution.

- (3) TOPSIS state yields compound gains over any isolated component. SAC-S1 shows that manually compacting the state to 6D (without TOPSIS) already yields +23.8 pp over SAC-Base, confirming that feature engineering matters. SAC-T1 shows that using TOPSIS to build the 6D state (instead of ad-hoc features) raises this to +27.6 pp, even when the action space still has 3 classes. Complete SAC-TOPSIS, combining TOPSIS state with binary action and deadline reward, achieves +39.9 pp, more than any individual component, confirming that the three design choices are synergistic rather than merely additive.

6 Conclusion and Future Work

This article presented SAC-TOPSIS, a hybrid architecture for deadline-aware task offloading in VEC networks. The central innovation is using TOPSIS not as an autonomous decision-maker, but as a state engineering module that converts a variable-sized, multi-criteria set of candidates into a fixed, scale-invariant 6D representation. A maximum entropy SAC agent trained on this representation learns a binary local/offloading policy that: (i) achieves a deadline fulfillment rate of 44.36%, the highest among all DRL policies and heuristics evaluated across 601 test scenarios; (ii) reaches an SWQ index of 0.1450; (iii) generalizes zero-shot to unseen vehicular densities (10–30 tasks/s) without retraining; and (iv) avoids the mode collapses of PPO-TOPSIS (always-local) and SAC-Base (always-V2V).

A cross-family ablation study confirms that TOPSIS state compression is the primary driver of performance, as it improves deadline fulfillment for all tested DRL families: SAC (+39.9 pp), TD3 (+1.4 pp), and PPO (+0.3 pp), regardless of their base learning algorithm. The residual advantage of SAC over TD3 (+1.35 pp) is a side effect of entropy-regularized exploration, which prevents the policy from collapsing into the conservative always-local strategy that affects off-policy deterministic methods at higher densities.

Future work.

We plan to extend the TOPSIS module to Fuzzy-TOPSIS, incorporating linguistic uncertainty into the criteria weights to handle delayed telemetry and noisy channel estimates. We will also investigate multi-agent extensions where each vehicle in $\mathcal{V}(t)$ runs its own SAC-TOPSIS agent with decentralized coordination. Finally, hardware-in-the-loop evaluation with real 5G sidelink radios is planned to validate the energy model under physical channel conditions.

References

- [1] Kandali, K., Nouh, S., Bennis, L., Bennis, H.: A comprehensive survey on vanet-iot integration toward the internet of vehicles: Architectures, communications, and system challenges. *Data and Metadata* **4**, 521 (2025) <https://doi.org/10.56294/dm2025521>
- [2] Uddin, A., Zhang, N., Sakr, A.H.: Intelligent offloading in vehicular edge computing: A comprehensive review of deep reinforcement learning approaches and architectures. *arXiv preprint arXiv:2502.06963v2* (2025)
- [3] Zheng, K., Zheng, Q., Chatzimisios, P., Xiang, W., Zhou, Y.: Heterogeneous vehicular networking: A survey on architecture, challenges, and solutions. *IEEE Communications Surveys & Tutorials* **17**(4), 2377–2396 (2015) <https://doi.org/10.1109/COMST.2015.2440103>
- [4] Naik, G., Choudhury, B., Park, J.-M.J.: IEEE 802.11bd & 5g nr v2x: Evolution of radio access technologies for v2x communications. *IEEE Access* **7**, 70113–70128 (2019) <https://doi.org/10.1109/ACCESS.2019.2918458>
- [5] Chen, S., al.: Vehicle-to-everything (v2x) services: Mode 4 communication in lte-v2x. *IEEE Access* **5**, 22461–22471 (2017)
- [6] Federal Communications Commission (FCC): Use of the 5.850-5.925 ghz band: Second report and order. Technical Report FCC 24-110, FCC (November 2024). Adopted November 20, 2024. Establishing sunset for DSRC by December 2025.
- [7] Mao, Y., You, C., Zhang, J., Huang, K., Letaief, K.B.: A survey on mobile edge computing: The communication perspective. *IEEE Communications Surveys & Tutorials*

- 19(4), 2322–2358 (2017) <https://doi.org/10.1109/COMST.2017.2745201>
- [8] Miriyala, S., Chirra, V.R.: Deep learning approaches for computation offloading in edge computing: A critical review. *Telecommunication Systems* **89**(42) (2026) <https://doi.org/10.1007/s11235-026-01426-y>
 - [9] Liu, Y., Yu, H., Xie, S., Zhang, Y.: Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks. *IEEE Transactions on Vehicular Technology* **68**(11), 11158–11168 (2019) <https://doi.org/10.1109/TVT.2019.2935450>
 - [10] Fang, J., Shi, J., Lu, S., Zhang, M.: An efficient computation offloading strategy with mobile edge computing for iot. *Micromachines* **12**(2), 204 (2021) <https://doi.org/10.3390/mi12020204>
 - [11] Long, Y., Wang, Z., Lan, S., Zhang, R., Xu, K.: Energy-latency tradeoff for task offloading and resource allocation in vehicular edge computing. *Computer Networks* **258**, 111026 (2025)
 - [12] Luo, Q., Luan, T.H., Shi, W., Fan, P.: Heterogeneous task oriented data scheduling in vehicular edge computing via deep reinforcement learning. *IEEE Transactions on Vehicular Technology* **73**(12), 19582–19596 (2024) <https://doi.org/10.1109/TVT.2024.3444815>
 - [13] He, Y., Ren, J., Yu, G., Cai, Y.: D2D Communications Meet Mobile Edge Computing for Enhanced Computation Capacity in Cellular Networks. *IEEE Transactions on Wireless Communications* **18**(3), 1750–1763 (2019) <https://doi.org/10.1109/TWC.2019.2896999>
 - [14] Wang, M., Yi, H., Jiang, F., Lin, L., Gao, M.: Review on offloading of vehicle edge computing. *Journal of Artificial Intelligence and Technology* **2**(4), 132–143 (2022) <https://doi.org/10.37965/jait.2022.0120>
 - [15] Wang, B., Wang, C., Huang, W., Song, Y., Qin, X.: A survey and taxonomy on task offloading for edge-cloud computing. *IEEE Access* **8**, 186080–186101 (2020)
 - [16] Chen, S., Li, W., Sun, J., Pace, P., He, L., Fortino, G.: An efficient collaborative task offloading approach based on multi-objective algorithm in mec-assisted vehicular networks. *IEEE Transactions on Vehicular Technology* **74**(7), 11249–11263 (2025) <https://doi.org/10.1109/TVT.2025.3543412>
 - [17] Ahmed, M., *et al.*: A survey on vehicular task offloading: Classification, issues and future challenges. *Journal of King Saud University – Computer and Information Sciences* **34**(7), 4135–4162 (2022) <https://doi.org/10.1016/j.jksuci.2022.05.016> . Elsevier / ScienceDirect
 - [18] Li, Z., Zhu, Q.: Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing. *Information* **11**(2), 83 (2020)
 - [19] You, Q., Tang, B.: Efficient task offloading using particle swarm optimization algorithm in edge computing for industrial internet of things. *Journal of Cloud Computing* **10**(1), 41 (2021)
 - [20] Kishor, A., Chakarbarty, C.: Task offloading in fog computing for using smart ant colony optimization. *Wireless personal communications* **127**(2), 1683–1704 (2022)
 - [21] Wu, Z., Jia, Z., Pang, X., Zhao, S.: Deep reinforcement learning-based task offloading and load balancing for vehicular edge computing. *Electronics* **13**(8), 1511 (2024)
 - [22] Cao, C., Guo, Z., Zheng, X.: A survey on task offloading decision-making algorithms in heterogeneous vehicular network. *IEEE Access* **13**, 1–20 (2025)
 - [23] Shao, M., *et al.*: Graph neural network-based task offloading and resource allocation for scalable vehicular networks. *IEEE Transactions on Vehicular Technology* (2025). Early Access / Preprint available 2024
 - [24] Shi, W., Chen, L., Zhu, X.: Task offloading decision-making algorithm for vehicular edge computing: A deep-reinforcement-learning-based approach. *Sensors* **23**(17), 7595 (2023)
 - [25] Li, Y., *et al.*: Deep reinforcement learning for task offloading in vehicular edge computing: A survey. *arXiv preprint arXiv:2501.01234* (2025). Foco em modelagem temporal e dinâmica de VEC
 - [26] Sousa Vieira, A.S., Souza, A.B., Celestino Júnior, J.: FOF: an energy-efficient task offloading in VEC-enabled vehicular networks using fuzzy TOPSIS. *Cluster Computing* (2025) <https://doi.org/10.1007/s10586-025-05027-3>
 - [27] Tan, L.T., Hu, R.Q.: Mobility-aware edge caching and computing in vehicle networks: A

- deep reinforcement learning. *IEEE Transactions on Vehicular Technology* **67**(11), 10190–10203 (2018)
- [28] Pang, H., Wang, Z.: Dueling double deep q network strategy in mec for smart internet of vehicles edge computing networks. *Journal of Grid Computing* **22**, 1–12 (2024)
- [29] Fofana, N., Letaifa, A.B., Rachedi, A.: Intelligent task offloading in vehicular networks: A deep reinforcement learning perspective. *IEEE Transactions on Vehicular Technology* (2024)
- [30] Gao, Z., Yang, L., Dai, Y.: Large-scale computation offloading using a multi-agent reinforcement learning in heterogeneous multi-access edge computing. *IEEE Transactions on Mobile Computing* **22**(6), 3425–3443 (2023)
- [31] Zheng, J., Zhang, Y., Luan, T.H., Mu, P.-K., Li, G., Dong, M., Wu, Y.: Digital twin enabled task offloading for iovs: A learning-based approach. *IEEE Transactions on Network Science and Engineering* **11**, 659–672 (2023)
- [32] Nie, L., Wang, H., Feng, G., Sun, J., Lv, H., Cui, H.: A deep reinforcement learning assisted task offloading and resource allocation approach towards self-driving object detection. *Cloud Computing* **12**, 131 (2023)
- [33] Liu, Z., Huang, L., Gao, Z., Luo, M., Hosseinalipour, S., Dai, H.: Ga-drl: Graph neural network-augmented deep reinforcement learning for dag task scheduling over dynamic vehicular clouds. *IEEE Transactions on Network and Service Management* **21**(4), 4226–4242 (2024)
- [34] Guo, H., et al.: Deep reinforcement learning for task offloading in vec. *IEEE Transactions on Mobile Computing* (2024)
- [35] Silva, V.R.G.d., Valderrama, C., Manneback, P., Xavier-de-Souza, S.: Analytical energy model parametrized by workload, clock frequency and number of active cores for share-memory high-performance computing applications. *Energies* **15**, 1213 (2022) <https://doi.org/10.3390/en15031213>
- [36] Rao, R., Vrudhula, S.: Performance optimal processor throttling under thermal constraints. In: *Proceedings of the 2007 International Conference on Compilers, Architecture, and Synthesis for Embedded Systems*, pp. 257–266 (2007). <https://doi.org/10.1145/1283340.1283383>
- [37] Study on channel model for frequencies from 0.5 to 100 ghz (3gpp tr 38.901 version 16.1.0 release 16). Technical Report ETSI TR 138 901 V16.1.0, 3GPP / ETSI (November 2020). Release 16
- [38] Yan, J.: Towards dependable 5g-nr sidelink communication. Ph.d. dissertation, Sorbonne Université (December 2024). HAL Id: tel-05077398
- [39] Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *International Conference on Machine Learning*, pp. 1861–1870 (2018). PMLR